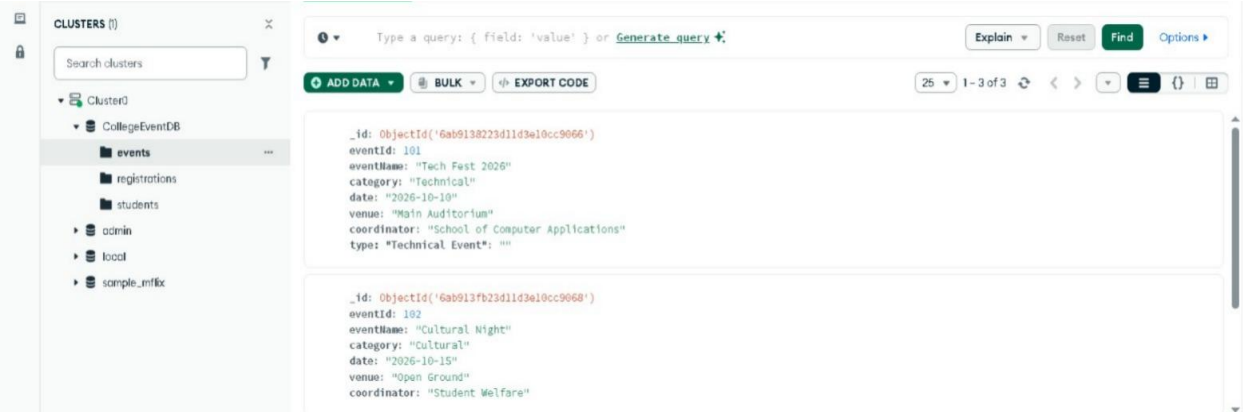
1. CREATE DATABASE

MongoDB Query:

use CollegeEventDB

Explanation: Creates or switches to the CollegeEventDB database.



2. CREATE COLLECTION

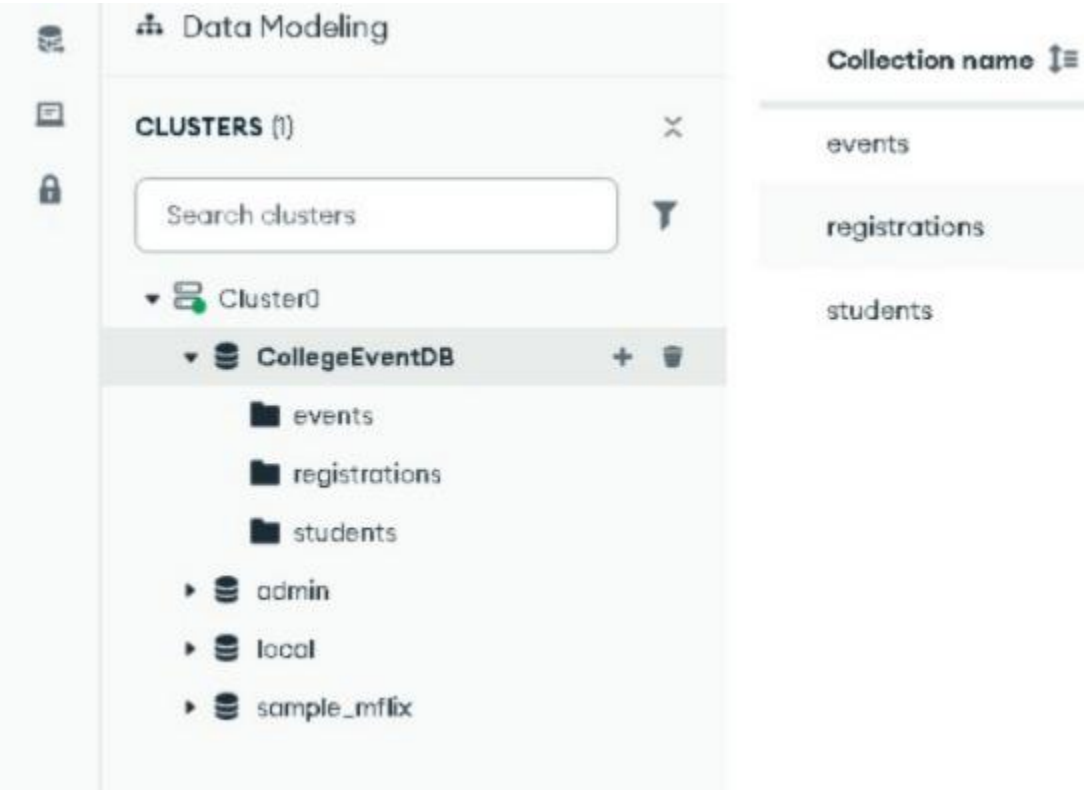
MongoDB Query:

db.createCollection("events")

db.createCollection("students")

db.createCollection("registrations")

Explanation: Creates three collections named events, students, and registrations for storing event, student, and registration information.

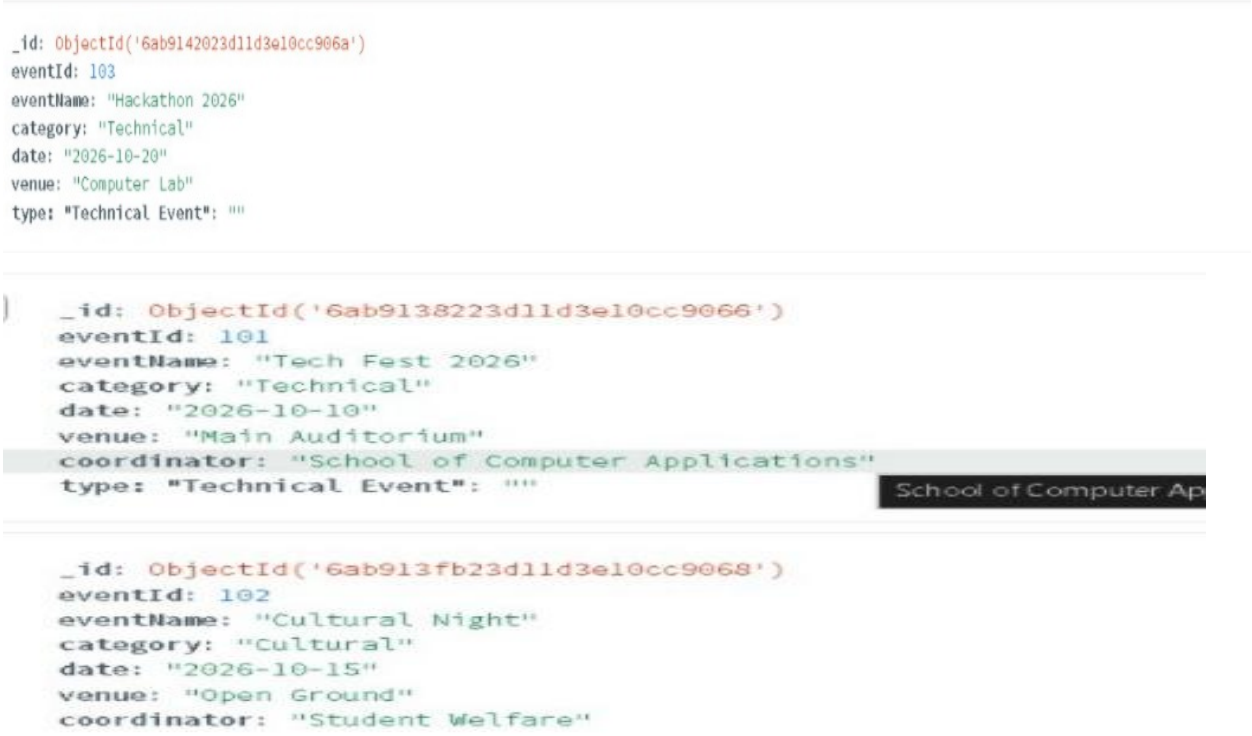


3.DISPLAY ALL EVENTS

MongoDB Query:

db.events.find()

Explanation: Retrieves all documents from the events collection.



4.EQUAL TO — $eq

MongoDB Query:

db.events.find({category: {$eq: "Cultural"}})

Explanation: Finds events whose category is exactly equal to Cultural.



5.GREATER THAN — $gt

MongoDB Query:

db.events.find({eventId: {$gt: 102}})

Explanation: Finds events whose eventId is greater than 102.

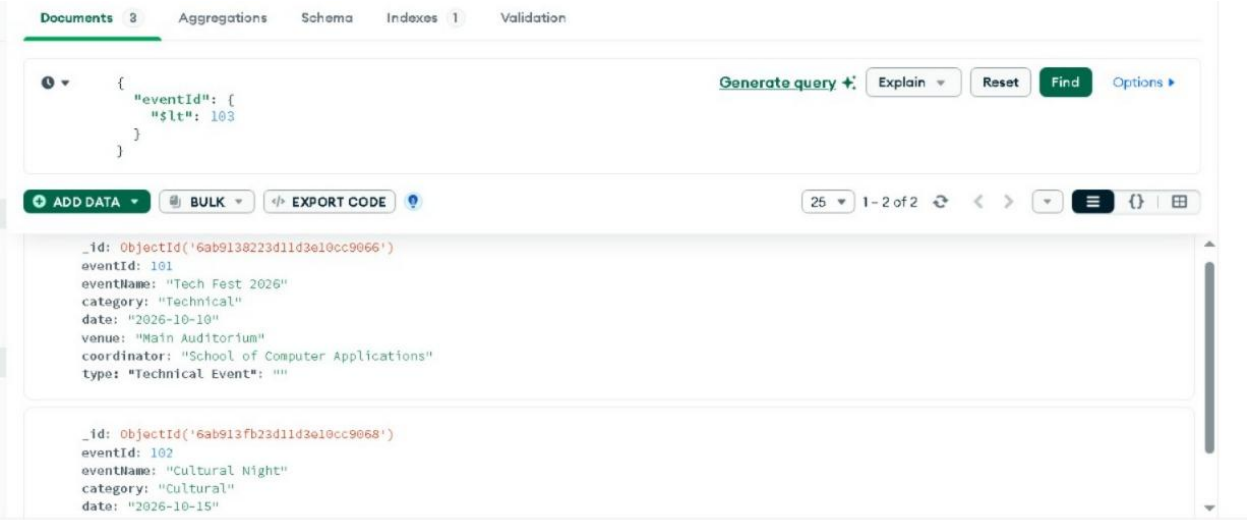


6.LESS THAN — $lt

MongoDB Query:

db.events.find({eventId: {$lt: 103}})

Explanation: Finds events whose eventId is less than 103.



7.GREATER THAN OR EQUAL TO — $gte

MongoDB Query:

db.events.find({eventId: {$gte: 103}})

Explanation: Finds events whose eventId is greater than or equal to 103.

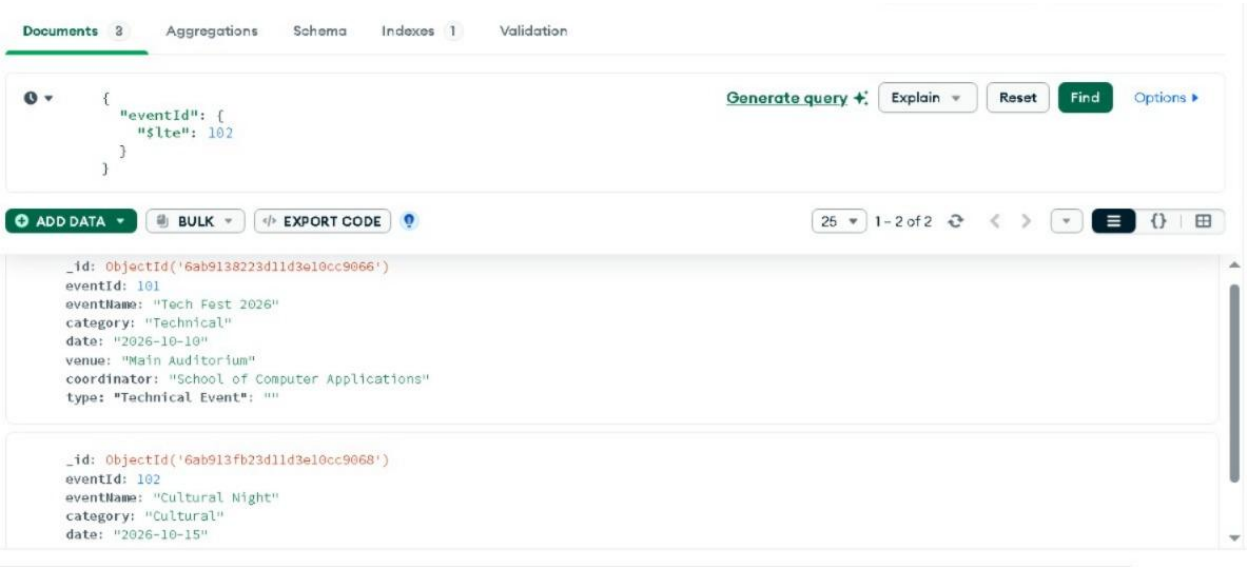


8.LESS THAN OR EQUAL TO — $lte

MongoDB Query:

db.events.find({eventId: {$lte: 102}})

Explanation: Finds events whose eventId is less than or equal to 102.

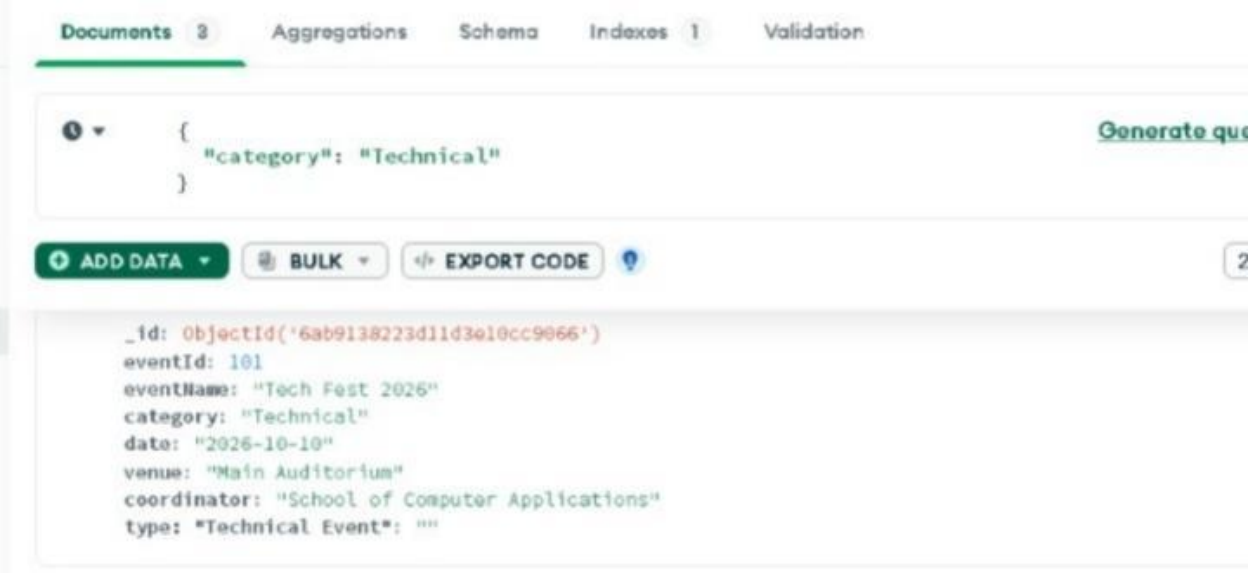


9.UPDATE ONE OPERATION

MongoDB Query:

```
db.events.updateOne(
{eventId: 101},
{$set: {venue: "Main Auditorium"}}
)
```

Explanation: Updates the venue to Main Auditorium of the event with eventId 101.

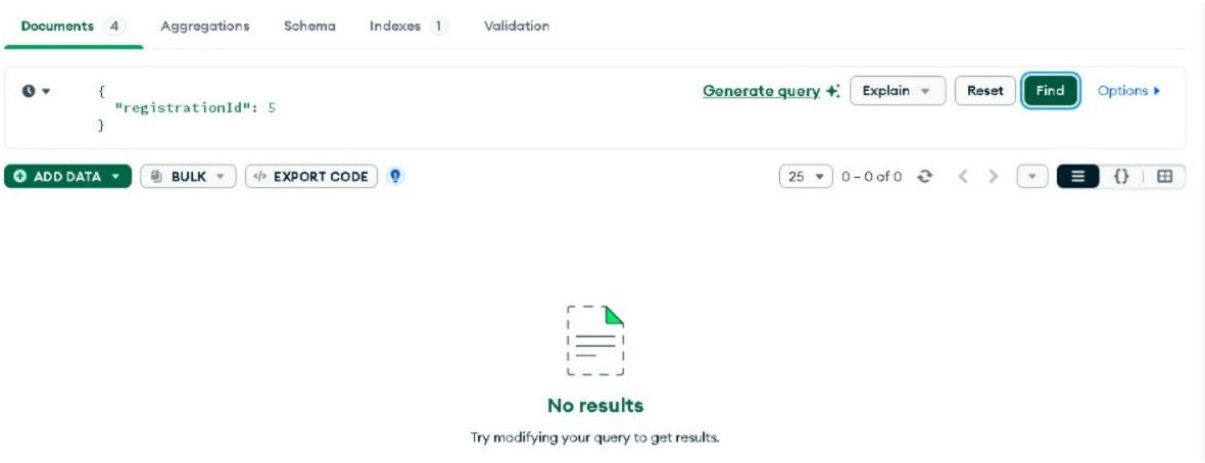


10.DELETE ONE OPERATION

MongoDB Query:

db.registrations.deleteOne({registrationId: 5})

Explanation: Deletes one registration document matching registrationId 5.

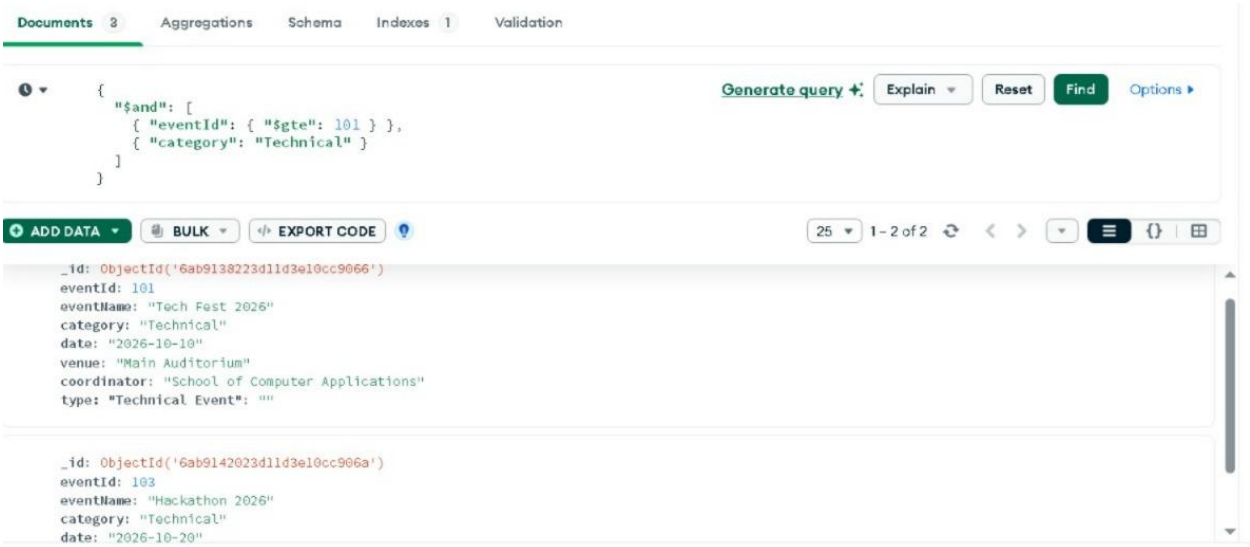


11.AND OPERATION — $and

MongoDB Query:

```
db.events.find({
$and: [
{eventId: {$gte: 101}},
{category: "Technical"}
]
})
```

Explanation: Finds events satisfying both conditions.



12.OR OPERATION — $or

MongoDB Query:

```
db.events.find({
```

$or: [

{category: "Cultural"},

{category: "Creative"}

]

})

Explanation: Finds events belonging to either the Cultural or Creative category.



MONGODB OPERATORS USED

Operator	Meaning	Example
$eq	Equal to	{ "age": { "$eq": 20 } }

$gt	Greater than	{ "marks": { "$gt": 70 } }
$lt	Less than	{ "age": { "$lt": 20 } }
$gte	Greater than or equal to	{ "marks": { "$gte": 60 } }
$lte	Less than or equal to	{ "marks": { "$lte": 60 } }
$and	All given conditions must be true	{ "$and": [{ "age": 20 }, { "city": "Lucknow" }] }
$or	At least one condition must be true	{ "$or": [{ "category": "Cultural" }, { "category": "Creative" }] }

CONCLUSION

The Student Management System was successfully created using MongoDB. Various MongoDB operations were performed on the data, including creating a database and collections, inserting and displaying documents, updating and deleting records, and using query operators such as $eq, $gt, $lt, $gte, $lte, $and, and $or.

This project helped in understanding the basic concepts of NoSQL databases and how MongoDB can be used to store, manage, search, update, and delete data efficiently. It also provided practical knowledge of MongoDB queries and different operators used for retrieving specific records.